MICROELECTRÓNICA

Modelo de Fallos



MsC. Luz Adanaqué Infante



MODELO DE FALLOS

- Es un conjunto de fallos seleccionados como los más importantes del circuito.
- La definición del modelo de fallos depende del nivel al que nos movamos (lógico, eléctrico, etc) de la tecnología en la que se trabaje y de la seguridad que se desea alcanzar con el test.
- La cobertura de fallos de un conjunto de vectores de test es un porcentaje de fallos del modelo escogido que se detectan al aplicar dichos vectores de test.
- Una cobertura por debajo del 95% es considerada como mala y el fabricante se negará a fabricar.
- El modelo debe ser preciso (los fallos modelados se deben acercar a la realidad)
- El modelo debe ser tratable (los fallos deben detectarse aún cuando el circuito sea grande)
- Ambos requerimientos son contradictorios.

MODELO DE FALLOS

MODELOS A NIVEL DE TRANSISTOR O INTERRUPTOR

- Cortocircuitos.
- Retardos.
- Acoplos entre nodos.
- Degradación de elementos.

MODELO DE FALLOS A NIVEL FUNCIONAL

- Simulan el comportamiento funcional de cada uno de los elementos de un circuito en presencia de defectos, para puertas lógicas es un modelo simple, mientras que para bloques funcionales no.

MODELOS A NIVEL LÓGICO

- Stuck -at (bloqueos)
- Bridges (puenteos)

MODELO DE STUCK -ATs

Los defectos físicos pueden modelarse como líneas del circuito lógico que quedan bloqueadas permanentemente a 0 o a 1.

Si tenemos una NAND de 3 entradas, la tabla muestra los vectores de test necesarios para detectar cualquier fallo simple de stuck -at de las entradas o la salida.

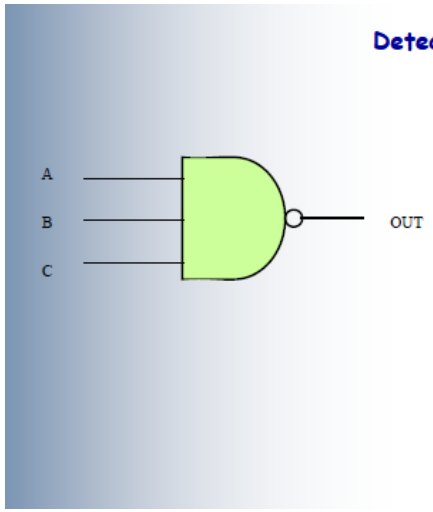
Al trabajar con tecnología NMOS, se muestran a trazos dos posibles “shorts” de líneas

¿Cuándo se detecta el fallo 1?

011

¿Cuándo se detecta el fallo 2?

111

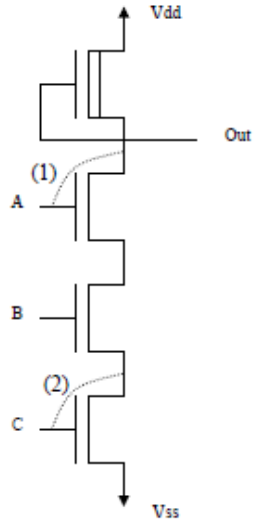


Detección de fallos stuck-at en una puerta NAND

A	B	C	F0	F1	F2
1	1	1	0	1	0
0	1	1	1	1	0
1	0	1	1	1	1
1	1	0	1	1	1

A,B y C entradas del circuito
F0: salida correcta del circuito sin fallos
F1: salida (A stuck-at 1
F2: salida (A stuck-at 0)

El primer vector permite detectar el fallo (A stuck-at 0) mientras que el segundo detecta el fallo (A stuck-at 1)



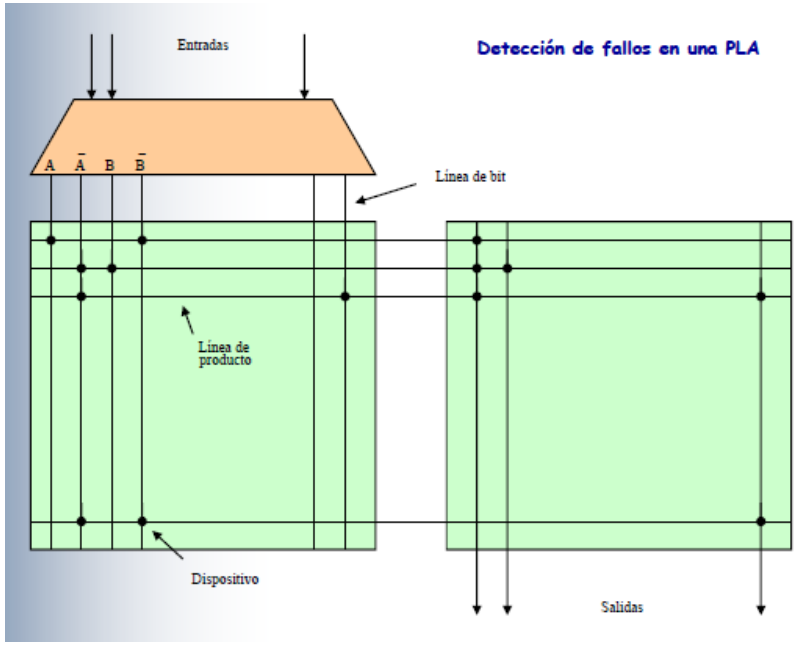
MODELO DE FALLOS EN BLOQUES FUNCIONALES (PLAs)

En los bloques funcionales podemos encontrar tres tipos de fallos:

- Los fallos de stuck: Una de las líneas queda fijada de manera permanente a uno de los dos valores lógicos.

- Los fallos de cruce: Ausencia o presencia extra de un dispositivo entre una línea de entrada y una de producto.

- Los fallos de puenteo (corte) es un corte entre dos líneas adyacentes o entre dos líneas que se cruzan.



Todos estos fallos se pueden detectar a nivel de la función lógica que implementa la PLA buscando si:

1) El fallo introduce términos producto que no existen en las funciones originales.
2) Introduce variables a algún término producto.
3) Elimina términos producto.
4) Elimina variables en algún término producto.

REDUCCIÓN DEL NÚMERO DE FALLOS

A	B	F	a/0	b/0	a/1	b/1
0	0	1	1	1	1	1
0	1	1	1	1	0	1
1	0	1	1	1	1	0
1	1	0	1	1	0	0

En la NAND, trabajando con stuck - at:

Si la entrada A, o B queda bloqueadas a 0, OUT = 1.

Entonces los fallos s-a-0 de A y B son fallos equivalentes.



Si la entrada A queda bloqueada a 1:

El fallo se propaga solo si : 0 se aplica a A, 1 a B, OUT debería ser 1 y sale 0 .

Dos fallos A y B son equivalentes cuando todos los patrones de test que detectan el fallo A detectan también el fallo B.

En cambio, un fallo A domina a un fallo B cuando cualquier patrón que detecta el fallo A detecta también el fallo B (pero no al contrario).

EJERCICIOS

Realizar el grafo y encontrar los fallos stuck at 0 y stuck at 1 de :

Y = A or B

Y = (A+B)C'(D+E)'

Y = (A+B+C)ABC

(completar la tabla)

A	B	F	a/0	b/0	a/1	b/1
0	0	1	1	1	1	1
0	1	1	1	1	0	1
1	0	1	1	1	1	0
1	1	0	1	1	0	0

DETERMINACIÓN DE VECTORES DE TEST

1) Se selecciona el modelo de fallos:

Si se desarrolla un ASIC, el fabricante es el que determina qué modelo utilizar, y si el circuito es digital, en el 99% de los casos este modelo será el de stuck - ats.
El fabricante obliga al diseñador a entregar el layout y un conjunto de vectores de test que garanticen un buen nivel de cobertura de fallos (95% - 98%).

2) La aplicación del modelo de fallos al circuito permite determinar la lista concreta de fallos a detectar.

3) Se buscan fallos equivalentes y redundantes en un intento de reducir el número de fallos.

4) Se procede a buscar, para cada fallo, un vector de test que lo detecte.

Para determinar los vectores se usa el MÉTODO DE SENSIBILIZACIÓN DE CAMINOS.



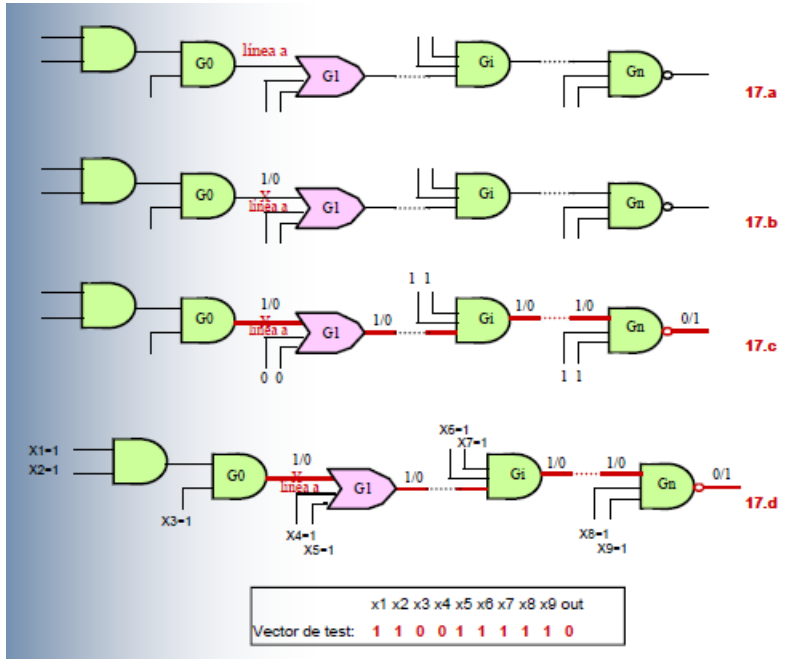
VECTORES DE TEST A NIVEL LÓGICO

1) Fase de set - up:

Si se quiere detectar el fallo (línea a stuck-at 0)
Cualquier combinación de los valores de entrada que aspire a detectar el fallo deberá forzar sobre la línea a el valor contrario al del fallo. De no ser así, los valores de salida que se producirán en caso de existencia o no de fallos, estos serán indistinguibles.

2) Fase de propagación:

Si la línea a genera un 1 en ausencia de fallo y 0 en presencia de él no sirve si no se ve desde el exterior. Entonces debe seleccionarse camino desde la línea a hacia una de las salidas del circuito, además de asignar valores a un mínimo número de líneas de este camino que garanticen la propagación del error a la salida.



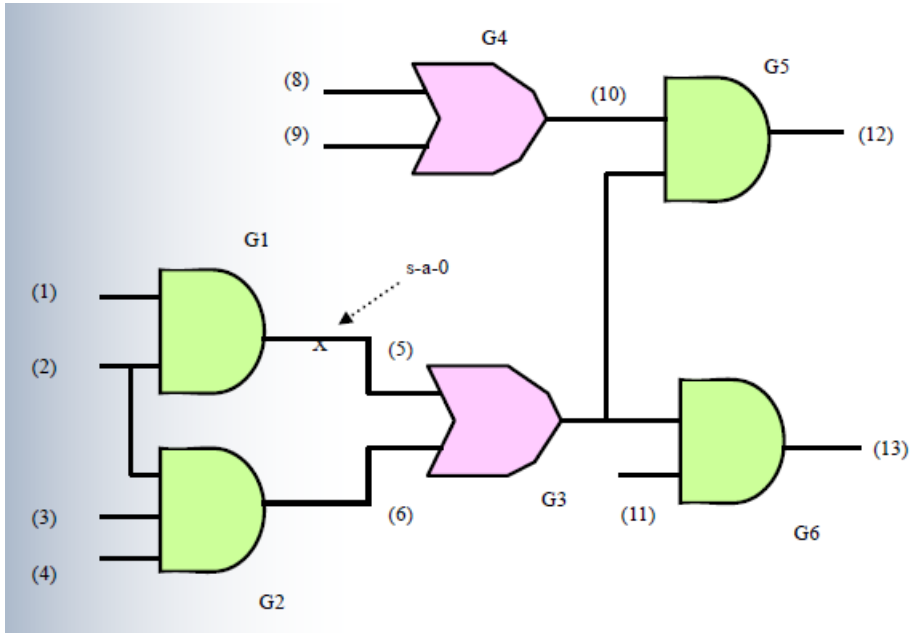
3) Fase de justificación: Restablecer valores sobre el resto de las líneas que garanticen que los valores establecidos en la fase de propagación se puedan alcanzar.

VECTORES DE TEST A NIVEL LÓGICO

El método de sensiblización de caminos no siempre consigue sus objetivos.
Existen fallos testeables para los cuales no se puede generar un vector de test si sólo se contempla 1 camino, como se muestra en la figura.

En el circuito, todos los caminos simples generan inconsistencias, sin embardo la combinación:

x1 = x2 = X3 = X4 = 1 detecta el fallo.

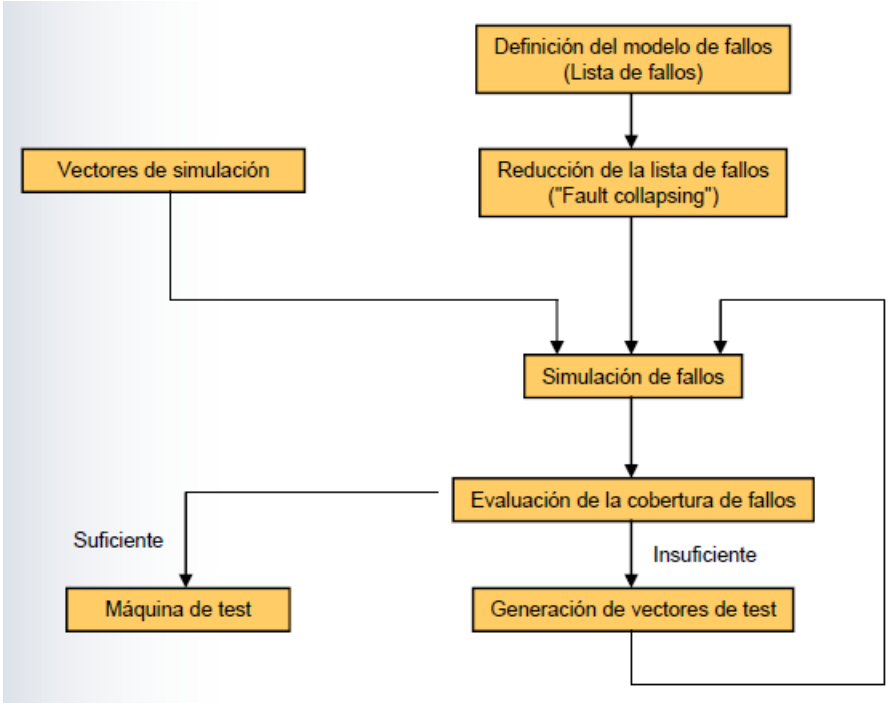


Para garantizar la detección del fallo, se debe considerar la sensibilización de todos los caminos simultáneamente.

Para ello existe una APROXIMACIÓN PRÁCTICA, que sería escoger los vectores de simulación y utilizarlos como vectores de test. Esto no alcanza una mayor cobertura, porque el diseñador sólo se basa en criterios netamente funcionales.

APROXIMACIÓN DE VECTORES TEST

- Decidir el modelo de fallos que se utilizará.
- Listar los fallos a detectar.
- Reducirlos mediante las equivalencias.
- A partir del conjunto de vectores de simulación se evalúa la cobertura de fallos, y si no es satisfactoria se utiliza un generador de vectores de test con el que se obtienen vectores que detectan algunos de los fallos.
- Cuando la cobertura es buena, el conjunto de vectores de test se considera conrrecto y se pasa por la máquina de test.



SIMULADORES DE FALLOS

- Es una herramienta CAD al que se le ingresa una descripción del circuito libre de fallos, un conunto de posibles fallos (lista de fallos), la especificación de las entradas y las salidas externas del circuito, y la secuencia de vectores test a simular.

- Para cada vector de test, el simulador simula la respuesta de la máquina libre de fallos, y la máquina en presencia de cada uno de los fallos posibles.

- Cuando a lo largo de la simulación se detectan valores distintos en las salidas externas para la máquina libre de fallos y para alguna de las máquinas erróneas, el fallo de ésta última se marca en la lista de fallos como detectado y la simulación de esta máquina se para.

Existen tres métodos de simulación de fallos:

- Simulación paralela.
- Simuación deductiva.
- Simulación concurrente.



DISEÑO PARA EL TEST

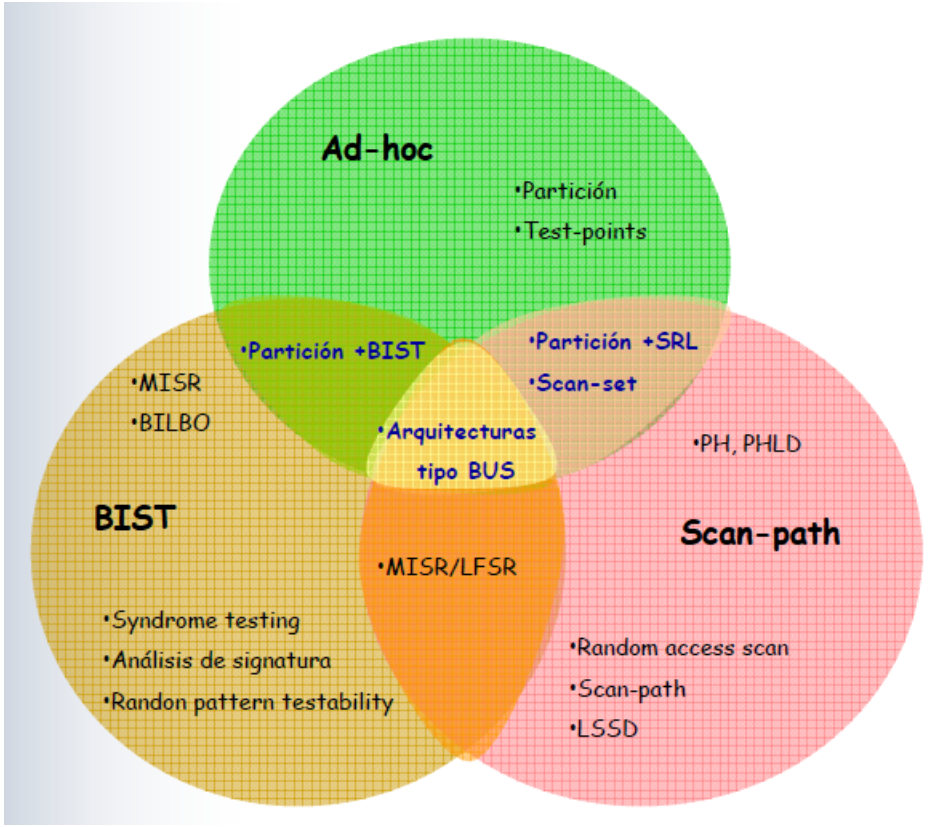
1. ¿Cómo se generan los vectores de test para las partes secuenciales del circuito?
Sabiendo que el método de sensibilización de caminos solo es aplicable al caso combinatorio.

2. ¿Cómo se implementan las técnicas de test concurrente y no - concurrente que permiten comprobar el circuito a lo largo de su vida activa?

3. ¿Con qué máquina de test se comprueban los circuitos que utilizan las tecnologías de punta?

El diseño para la testeabilidad inserta lógica adicional en el circuito para solucionar las fallas del mismo.

Una de las técnias es la llamada BIST (Built in Self Test)



TÉCNICAS DE DISEÑO PARA LA TESTEABILIDAD

- TÉCNICAS AD-HOC:

Resuelve los problemas de testeabilidad de un determinado diseño, no se pueden generalizar.

- TÉCNICAS ESTRUCTURADAS

Resuelven problemas generales con una metodología de diseño que utiliza un conjunto de reglas que garantizan que el circuito resultante es fácilmente testeable.

Una de ellas es del tipo SCAN - PATH : Combina normas de diseño para conseguir una modificación correcta de los latches que constituyen el sistema, con la reconexión de los mismos en una serie de registros de desplazamiento que el sistema se activa en modo test.

Otra es la Built in self Test (BIST), que genera y comprime pseudoaleatoriamente los vectores de test. A diferencia del scan path, estas técnicas no necesitan introducir ni observar los vectores del test al/del circuito. Solo se compara el valor guardado en el registro con el de la salida, y se sabe si habrá error o no.



TÉCNICAS DE SCAN PATH

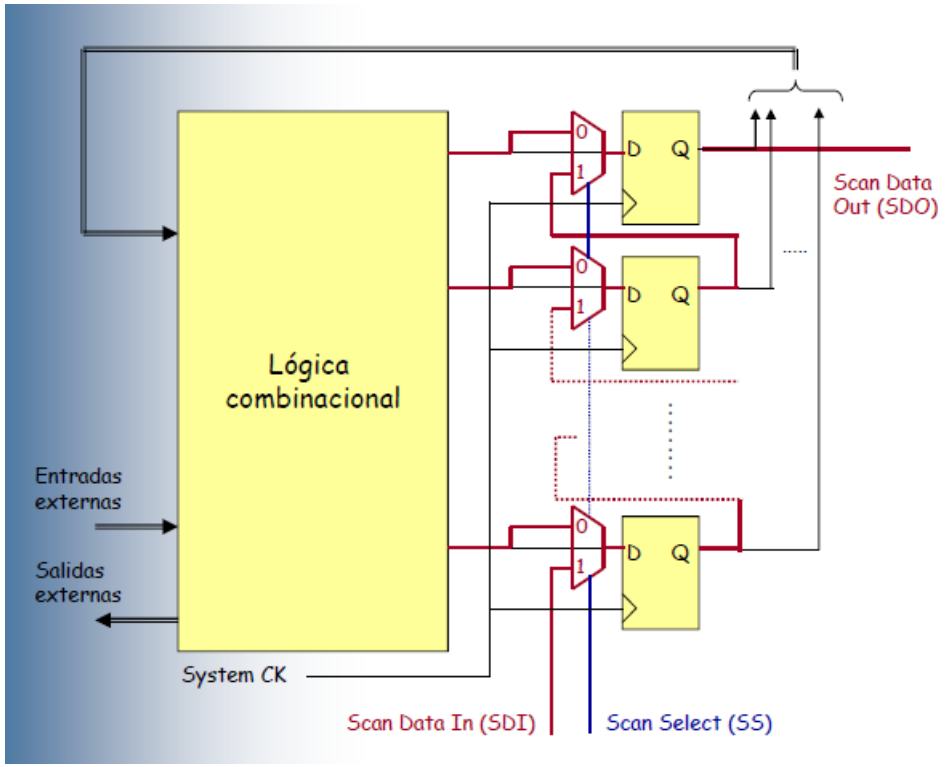
1. Generaremos el conjunto de vectores de es que comprueben el funcionamiento de la parte combinacional con una cobertura de fallos adecuada, tal como vimos anteriormente.

2. Test de la parte secuencial:

Scan - Select = 1 y testearemos el registro de desplazamiento formado a través de su entrada SDI y su salida SDO.

Una forma de testear el registro de desplazamiento con una cobertura de fallos adecuada es :
- Hacer un reset del registro.
- Pasar un 1 a lo largo de todo el registro, comprobando la salida.
- Pasar la secuencia 001100110011

3. Una vez comprobado el buen funcionamiento de los elementos de memoria, se ataca la parte combinacional.



VENTAJAS DEL SCAN PATH

1. Disociación lógica combinacional - secuencial.

Sólo es necesario generar vectores de test para la parte combinacional.

2. Permite la partición del circuito utilizando varios scan - paths.

3. Una forma sencilla de reducir el número de ciclos de test necesarios para comprobar la parte combinacional es distribuir los elementos de memoria en dos o más scan - paths.

4. Optimiza el uso de los simuladores de fallos y ATPGs, estos se pueden aplicar por separado a las partes de lógica combinacional “aisladas” por los distintos scan - paths que utilicemos, aumentando su velocidad.

5. Cierta capacidad de diagnóstico.

Hasta aHora hemos hablado de la capacidad de testear el funcionamiento del circuito, pero una vez identificada la presencia de un fallo, ¿se puede localizar? ¿Saber en dónde está? NO, pero podemos tener una idea.

DESVENTAJAS DEL SCAN PATH

1. Necesita lógica adicional (en general 5 - 15% del overhead de área debido a la inclusión de scan paths).

2. Necesidad de pines adicionales, Scan-select y tantos pines de Scan-Data-In y Scan-Data-Out como registros de desplazamiento hayamos incluido.

3. Se requiere de un elevado número de ciclos de test.

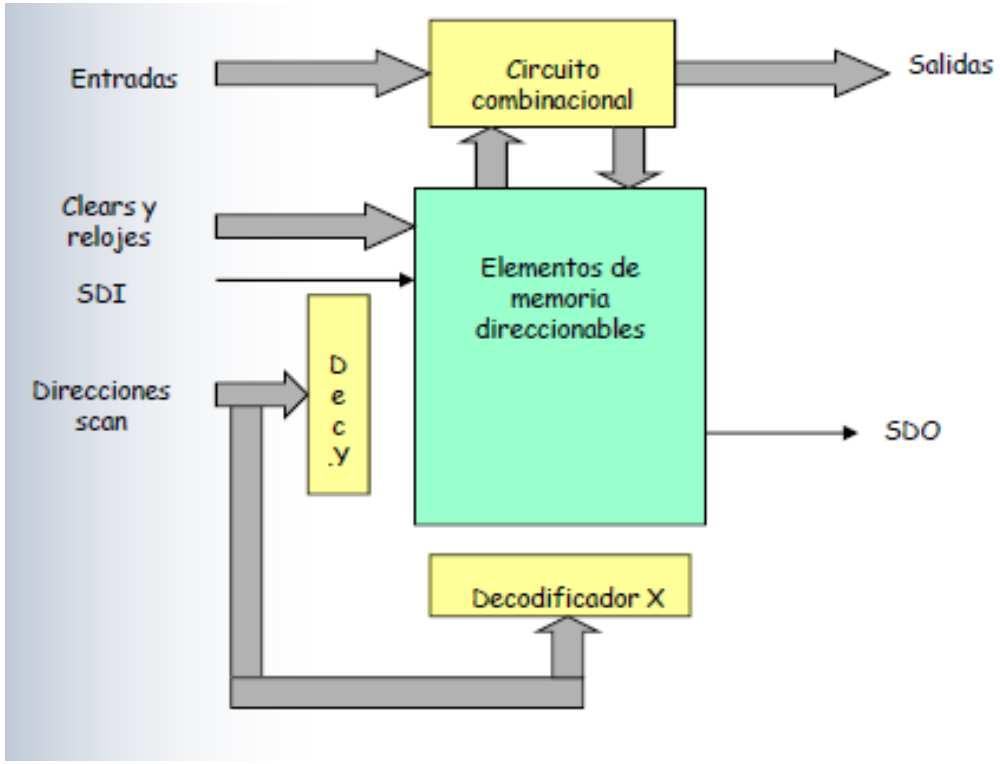
4. El circuito no se testea en sus condiciones de funcionamiento reales.

5. En general, se degradan las prestacione del circuito porque se incluye lógica que aumenta:

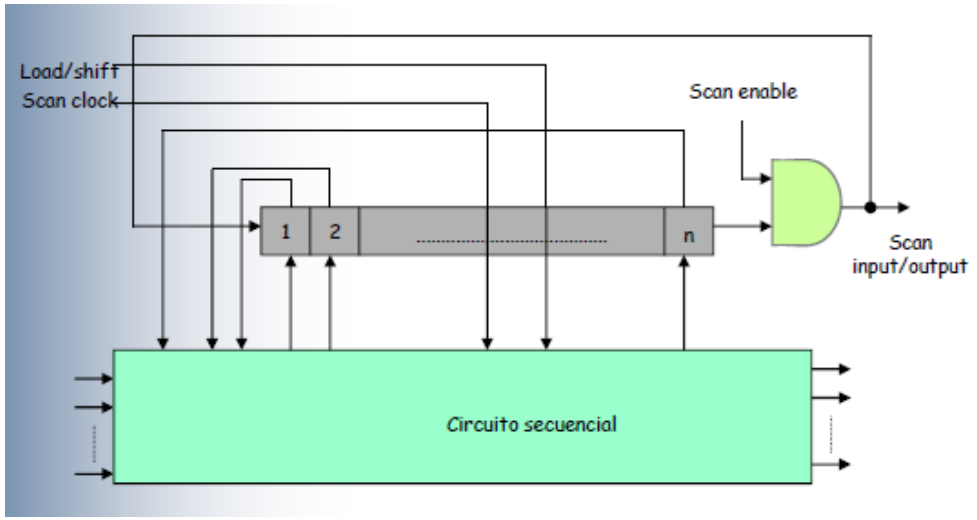
   Los retardos.
   El consumo.
   El riesgo de malfuncionamiento.

TÉCNICAS DE SCAN PATH

RANDOM ACCESS SCAN:



SCAN - SET LOGIC:



BIST : BUILT IN SELF TEST

Son técnias de test no-concurrente, donde se añadeuna lógica al circuito capaz de generar internamente los vectores de test, forzándolos sobre el circuito, y comrpobar los valores obtenidos con los esperados, todo ello sin necesidad de algún equipo externo.

Sus ventajas respecto a otras técnicas son:

1. Eliminan, o al menos reducen los costos debido a la generación de vectores de test y simulación de fallas.

2. El test de los circuitos se puede realizar la velocidad de operación normal del circuito.

3. Se simplifica el equipo externo necesario para comprobar circuitos.

Por otro lado:

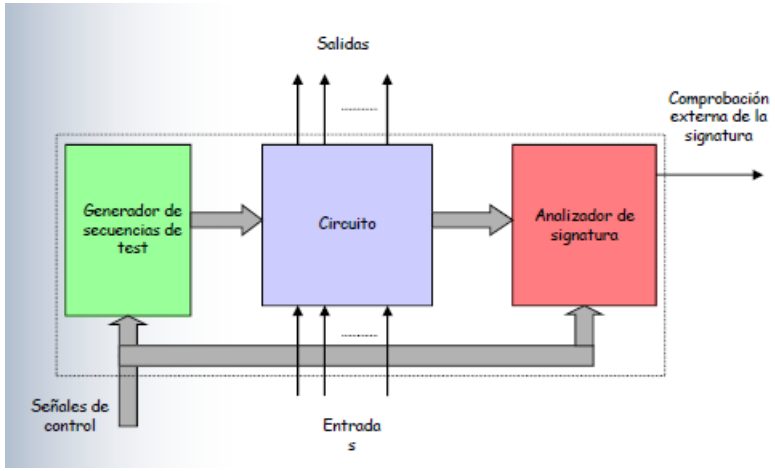
1. Necesita un espacio propio para la lógica adicional que ha de realizar la comprobación del circuito.
2. No se puede acceder ni observar nodos internos determinados como en el caso de las scan path.



BIST : BUILT IN SELF TEST

TEST EXHAUSTIVO

Aplica a los 2^N donde N es el número de entradas del circuito.
El generador puede ser, por lo tanto, cualquier módulo capaz de generar los 2^N combinaciones: Un contador y un generador de código.
Es posible aplicar el BIST a partes del circuito que si puedan testearse exhaustivamente.



TEST PSEUDOALEATORIO

Se basa en la aplicación de un subconjunto de las 2^N combinaciones de entradas ecogidos de una manera pseudoaleatoria. Por pseudoaleatoria se entiende una secuencia de combinaciones de los valores de entrada que, si bien una vez determinada se aplica siempre la misma secuencia, en su selección se han tenido en cuenta criterios matemáticos de aleatoriedad.

BIST : BUILT IN SELF TEST

- COMPROBADOR DE PARIDAD

  0: Si el número de unos de la secuencia de salida es par.
  1: Si es impar.

  Método fácil de implementar pero poco eficiente.

- ANALIZADOR POR CONTEO DE 1S

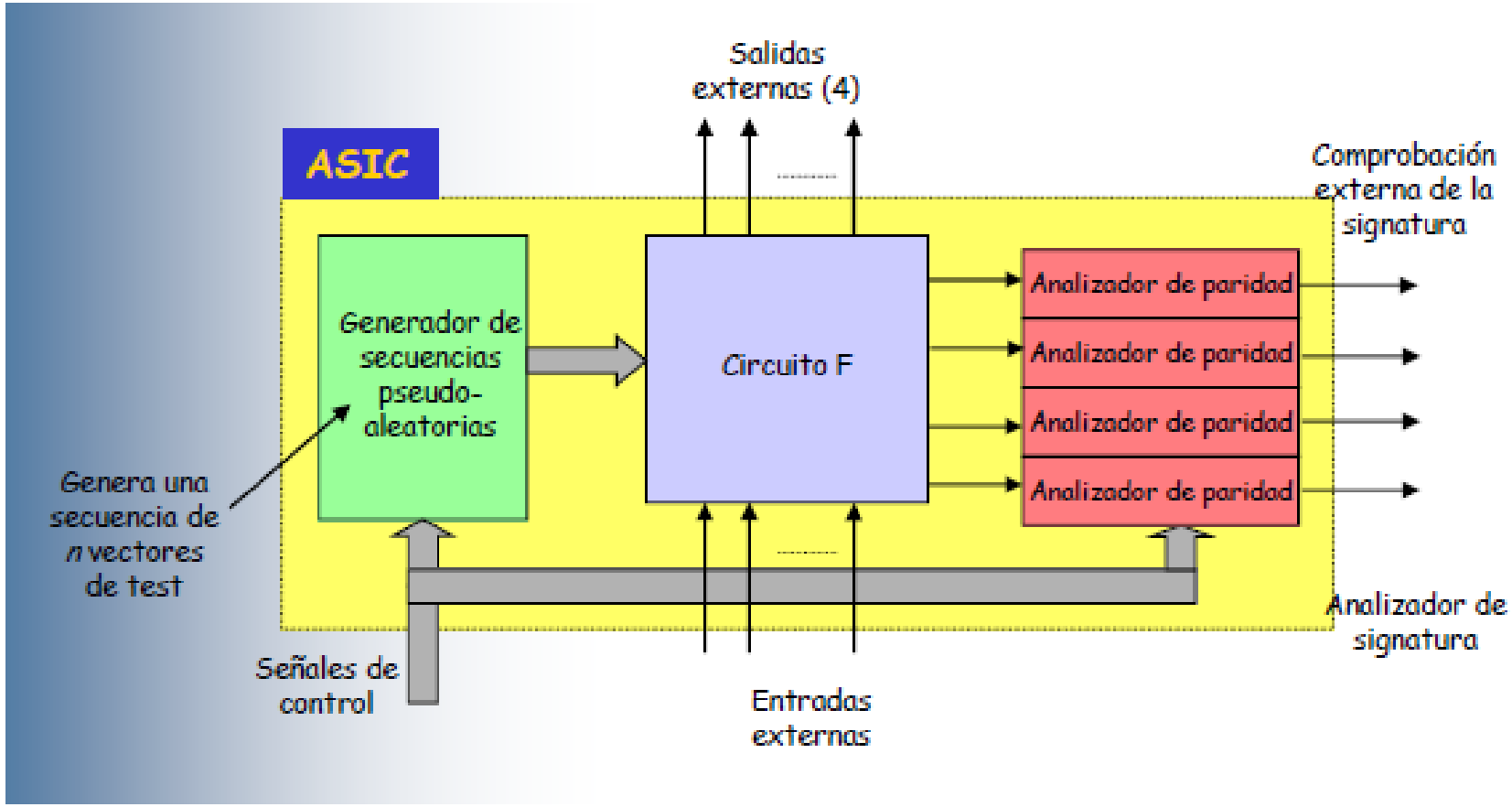
  La signatura es K cuando el número de 1s de la secuencia de salida es K.

- COMPRESIÓN POR TRANSICIÓN DE ESTADOS.

  Se cuentan el número de transiciones de 0 a 1, o de 1 a 0 que aparecen en una secuencia de N bits.
  Un circuito compresor muy simple es un contador de almenos $\log_2 N$ estados.



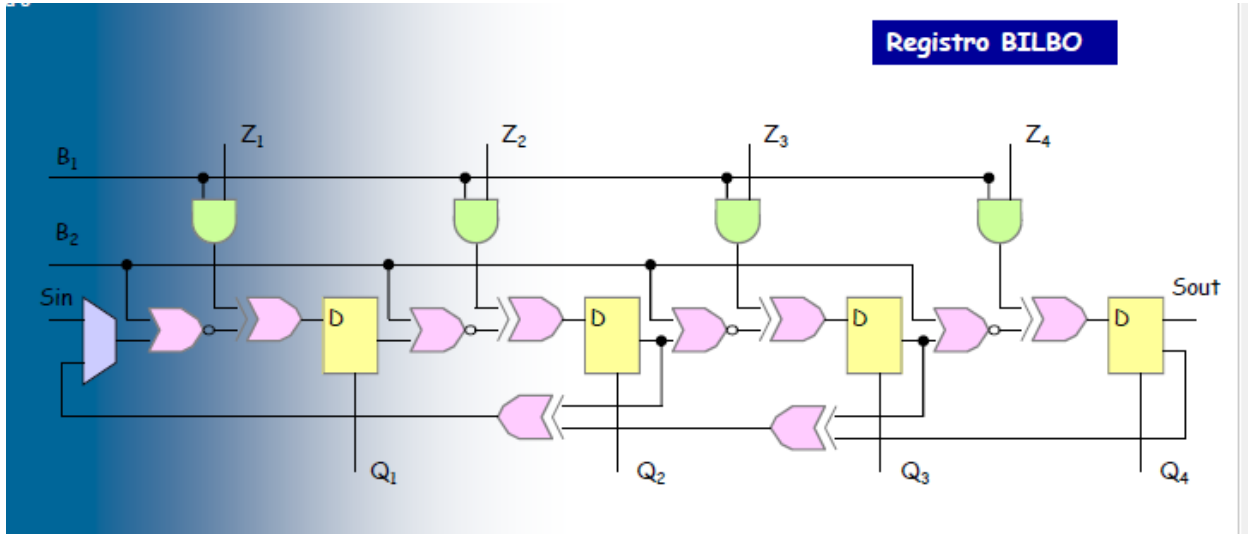
EJEMPLO DE UN BIST

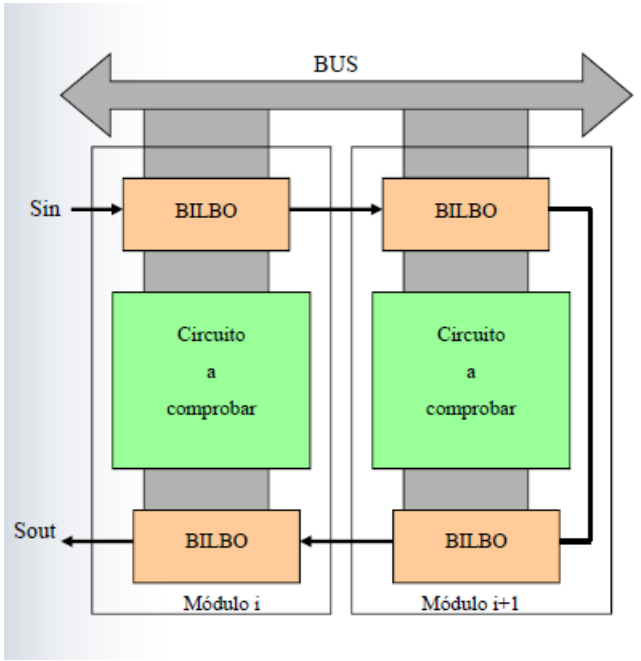


REGISTRO BILBO

Uno de los sistemas de test interno no - concurrente más popular es la técnica BILBO (Built- in Logic Block Observer).

En esta técnica se integran los conceptos del scan-design y de análisis de asignatura utilizando un LFSR al interno del circuito integrado que puede funcionar como analizar de signaturas y puede generar un conjunto de vectores de test en forma pseudoaleatoria.





REGLAS PARA EL DISEÑO CON TESTEABILIDAD

Regla 1: Facilitar el acceso y la observación de nodos internos

Regla 2: Asegurar la inicialización de los biestables (reset)

Regla 3: Evitar la lógica redundante

Regla 4: Evitar la generación interna de señales de reloj. No incluir circuitería asíncrona.

Regla 5: Partición del circuito

Regla 6: Documentar el diseño

… ect

